TRABALHO 1 · UNESC

Infraestrutura Cloud AWS

CRUD de produtos em Flask rodando em EC2, com banco PostgreSQL no Amazon RDS

Danielli Machado

16/10/2026

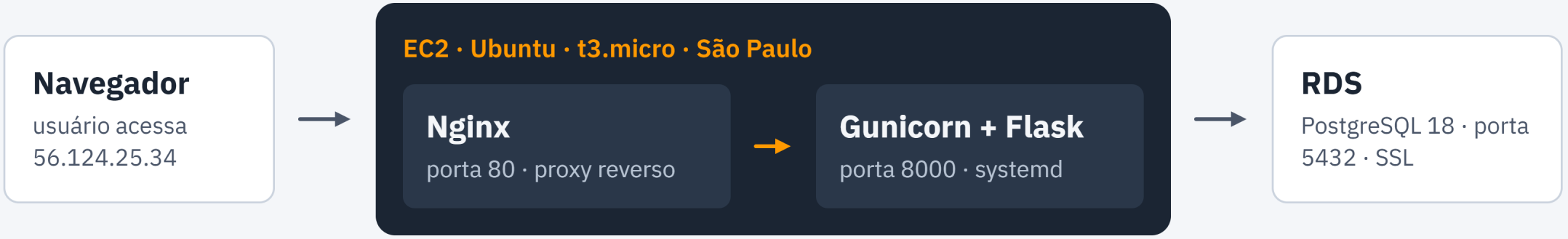
O QUE FOI PEDIDO

Todos os requisitos atendidos

Requisito do professor	Como foi feito
Aplicação CRUD	Produtos com estoque e vendas, em Python + Flask
VPS (EC2) com a stack	Ubuntu 26.04, Python, Nginx, Gunicorn, Git
Repositório de código	GitHub: DanielliMachado/crud-aws
Copiar e servir a aplicação	git clone no servidor + serviço systemd + Nginx
Banco no AWS RDS	PostgreSQL 18, privado, acessível só pela EC2
Configurar backup	Backup automático do RDS + snapshots + Git

ARQUITETURA

O caminho de uma requisição



</> **Código:** meu PC → **git push** → GitHub → **git clone / git pull** → EC2

VPS · AMAZON EC2

O servidor: crud-servidor

Configuração

- Sistema: Ubuntu Server 26.04 LTS
- Tipo: t3.micro (nível gratuito)
- Região: América do Sul (São Paulo)
- Disco: volume EBS gp3 de 8 GiB
- IP público: 56.124.25.34
- Acesso: par de chaves RSA (.pem)

Grupo de segurança (firewall)

22 SSH: administração, só com a chave

80 HTTP: o site, aberto para todos

Em produção, o SSH seria restrito a IPs conhecidos.

STACK NO SERVIDOR

Quatro peças rodando juntas

Python 3.14 + venv

Ambiente isolado com as bibliotecas do projeto, instaladas pelo requirements.txt

Flask

A aplicação: rotas para criar, listar, editar e excluir, com SQL parametrizado

Gunicorn + systemd

Servidor de produção como serviço: liga no boot e reinicia sozinho se cair

Nginx

Proxy reverso: recebe na porta 80 e repassa para o Gunicorn na 8000

REPOSITÓRIO E DEPLOY

Do meu PC para a nuvem pelo GitHub

1

Altero o código

no meu computador

2

git push

envia para o GitHub

3

git pull

o servidor baixa

4

restart

o serviço recarrega

```
cd ~/crud-aws && git pull && sudo systemctl restart crud
```

A senha do banco fica só no servidor, em variáveis de ambiente, e nunca vai para o GitHub.

BANCO DE DADOS · AMAZON RDS

PostgreSQL gerenciado e privado

Configuração

- Instância: crud-db
- Mecanismo: PostgreSQL 18
- Classe: db.t4g.micro (nível gratuito)
- Armazenamento: 20 GiB
- Tabelas: produtos, vendas, itens_venda

Segurança

- Acesso público: **Não**
- Porta 5432 liberada só para a EC2
- Conexão criptografada (SSL, TLS 1.3)
- Patches e hardware por conta da AWS

EVOLUÇÃO DO SISTEMA

Estoque e vendas com o sistema no ar

Campo quantidade

Coluna nova criada no RDS com ALTER TABLE, sem perder os produtos já cadastrados

Venda com vários itens

Tabelas vendas e itens_venda: uma venda tem vários itens (relação 1 para N)

Transação

Se faltar estoque de um item, nada é vendido: ou grava tudo, ou nada

Cada mudança seguiu o mesmo deploy: **commit** → **push** → **git pull** → **restart**, sem tirar o banco do ar.

BACKUP

Três camadas de proteção

Dados · RDS

Backup automático diário com restauração a qualquer minuto (point-in-time), mais um snapshot manual antes da apresentação

Servidor · EBS

Snapshot do disco da EC2: uma “foto” do servidor com a stack instalada e configurada

Código · GitHub

Cada commit é um ponto do histórico; o servidor pode ser recriado com um git clone

CONCEITOS DA AULA NA PRÁTICA

Onde cada conceito apareceu

Responsabilidade compartilhada

AWS: hardware e PostgreSQL. Eu: firewall, senhas, chaves e MFA

Uso medido

Nível gratuito e alerta de orçamento de gasto zero no Budgets

On-demand self-service

Servidor e banco criados pelo console em minutos, sem intermediários

Resiliência

Serviço que reinicia sozinho e backups para recuperar os dados

DEMONSTRAÇÃO AO VIVO

http://56.124.25.34

1. Cadastrar, editar e excluir um produto no site
2. Registrar uma venda com dois produtos e ver o estoque baixar
3. Conferir os dados direto no banco com um SELECT
4. Mostrar no console: EC2, RDS privado e os backups